Haoquan Wang
U1570975
2026-3-5

Assignment 7: GraphUtility

1. I invoked one of the three pairing program exemptions due to special circumstances. My teammate dropped out of the course after completing part of the assignment.

2.

3.

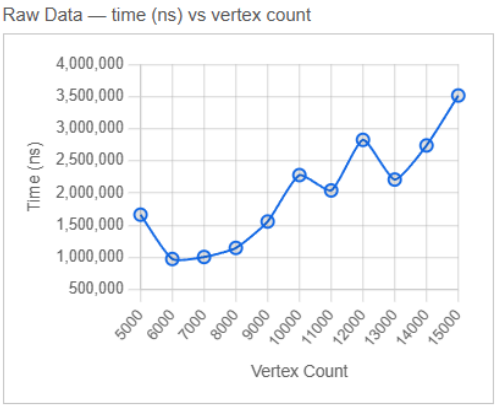
   a. CallGraphUtility.sort()IncludesMap building time(Iterate through the sources/destinations list, create Vertex and Edge objects), but the experiment only wants to measure the time of the sorting algorithm itself. Directly call...topoSort()The timing diagram has already been created in setupExperiment, and runComputation only calculates the sorting part, making the timing more straightforward.

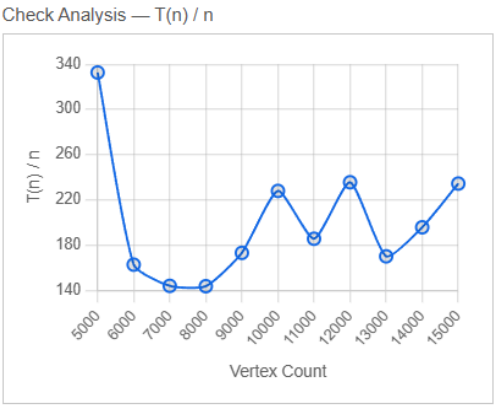
   b. random:Source VertexsourceVertexData = rng.nextInt(problemSize - 1)The target vertex is(sourceVertexData, problemSize)Random selection is made within a specified range to ensure randomness.

      Acyclic:Target vertex numberStrictly greater thanSource vertex number (nextInt(sourceVertexData + 1, problemSize)Since the edge only moves in the direction of the larger number, it is impossible to form a cycle.

      Sparse:Add only2 × problemSizeThe number of edges is far less than that of dense graphs.|V|²Magnitude.

   c. The theoretical complexity of topoSort is O(|V| + |E|). In a sparse graph, |E| = 2|V|, so O(|V| + 2|V|) = O(|V|) is linearly increasing. The ratio of T(n) / n tends to a constant as n increases, indicating that the growth rate of the running time meets the expectation of O(n²).



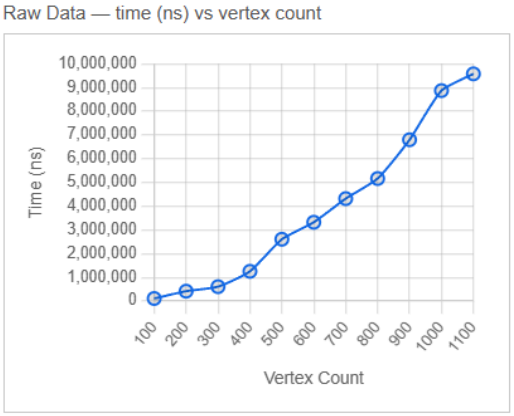


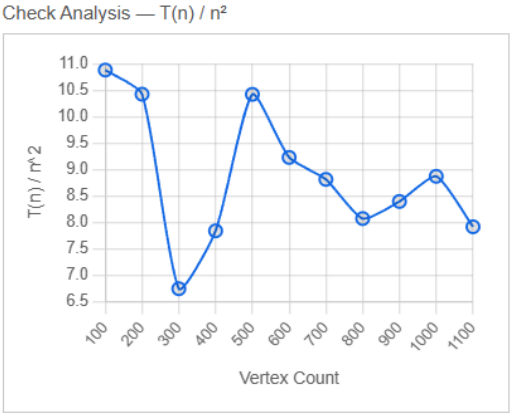
4.

a. Dense: Use a double loop: outer i from 0 to |V|-1, inner j from i+1 to |V|-1, add an edge i→j for each pair (i, j).

   The vertex range was reduced to 100-1100 because the number of edges is O(|V|²), which consumes a lot of memory. Having too many vertices could lead to an OutOfMemoryError.。

b. The theoretical complexity of topoSort is O(|V| + |E|). In dense graphs, |E| = |V|²/2, so O(|V|²) represents the squared growth. When the number of vertices is greater than 400, the ratio of T(n) / n² tends to a constant as n increases, indicating that the growth rate of the running time meets the expectation of O(n²).





5.

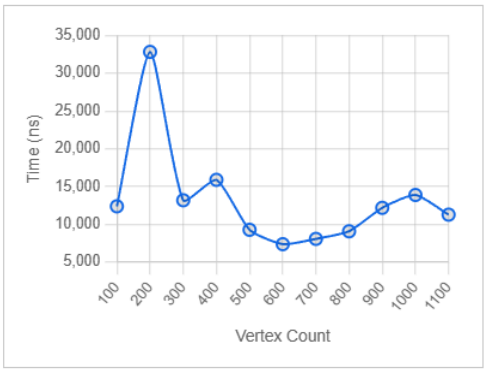
a. Graph settings:The requirement of acyclicity is no longer applied, and edges are completely randomized (both source and destination are randomly selected from [0, |V|)), simulating a real sparse graph. The number of edges remains 2×|V| to maintain sparsity.

   Vertex range:Because DFS is implemented recursively, a large number of vertices can cause a stack overflow, so we start with a smaller value.

   Source/Destination Selection:Fix source=0, destination=|V|-1, and force the addition of these two vertices to the graph in setupExperiment to ensure that DFS runs normally every time without throwing IllegalArgumentException.

b. The theoretical complexity of DFS is O(|V| + |E|). In a sparse graph, |E| = 2|V|, so O(|V|) grows linearly. The ratio of T(n) / n tends to a constant as n increases, indicating that the growth rate of the running time is consistent with the expected O(n²).

Raw Data — time (ns) vs vertex count



Check Analysis — T(n) / n